

Эффективная реализация анонимных функций в объектно-ориентированных языках

Болохононов Артем

Научный руководитель: Трепаков Иван Сергеевич

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами.

```
val capture = "arg = "  
  
val lambda = { x => println(capture + x) }  
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами.

```
class Lambda$$1(val prefix: Str) extends (Int ⇒ Unit) {  
  override def apply(x: Int) = {  
    println(prefix + x)  
  }  
}  
  
val capture = "arg = "  
val lambda = new Lambda$$1(capture)  
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами.

```
class Lambda$$1(val prefix: Str) extends (Int ⇒ Unit) {  
  override def apply(x: Int) = {  
    println(prefix + x)  
  }  
}  
  
val capture = "arg = "  
val lambda = new Lambda$$1(capture)  
lambda(42)
```

Таким образом, анонимные функции в объектно-ориентированных языках — это полноценный объект, который требует как аллокации, так и сборки мусора. Из-за чего возникают издержки по памяти и времени.

Постановка задачи

Цель: оптимизация издержек представления анонимных функций в объектно-ориентированных языках программирования.

Постановка задачи

Цель: оптимизация издержек представления анонимных функций в объектно-ориентированных языках программирования.

Задачи:

1. Изучить существующие подходы к уменьшению издержек на выделение объектов в куче,
2. Разработать подход к оптимизации издержек анонимных функций и функциональных замыканий,
3. Выполнить экспериментальную реализацию предложенного подхода,
4. Провести апробацию на представительном наборе тестов и приложений.

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = SomeObject(42, 42.0, "42")
useField(obj.a)
useField(obj.b)
obj.c = "override"

if (someBoolean) {
    method(obj)
}

if (anotherBoolean) {
    return obj
}
```

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

Межпроцедурный анализ частичных утеканий

Идея

```
def lambdaUse(x: (Int => Unit)) = {  
  ...  
  if (globalFlag) { // Всегда = false  
    globalVar = x  
  }  
  ...  
}  
  
val lambda = { x => println(x) }  
lambdaUse(lambda)
```

Межпроцедурный анализ частичных утеканий

Идея

```
def lambdaUse(x: (Int => Unit)) = {  
  ...  
  if (globalFlag) { // Всегда = false  
    globalVar = evacuate(x)  
  }  
  ...  
}  
  
val lambda = onStack( { x => println(x) } )  
lambdaUse(lambda)
```


Межпроцедурный анализ частичных утеканий

Идея

```
def lambdaUse(x: (Int => Unit)) = {  
  ...  
  if (globalFlag) { // Всегда = false  
    globalVar = evacuate(x)  
  }  
  ...  
}  
  
val lambda = onStack( { x => println(x) } )  
lambdaUse(lambda)
```

- Будем перемещать объект на стек, если у него нет гарантированного *утекающего* использования,
- Если такие использования есть, то перед их исполнением будем вставлять операцию *эвакуация*, которая создаст копию объекта на куче.

Межпроцедурный анализ частичных утеканий

Оптимизации

1. Быстрые пути в эвакуации

```
def evacuation[T](x: T): T = {  
  ...    // системный вызов  
}
```

Межпроцедурный анализ частичных утеканий

Оптимизации

1. Быстрые пути в эвакуации

```
def evacuation[T](x: T): T = {  
  if (x == null) {  
    return null  
  } else if (isOnHeap(x)) {  
    return x  
  } else {  
    ...    // системный вызов  
  }  
}
```

Межпроцедурный анализ частичных эвакуаций

Оптимизации

2. Дополнительный стековый слот для сохранения результата

```
val lambda = onStack( { x => println(x) } )
```

Межпроцедурный анализ частичных эвакуаций

Оптимизации

2. Дополнительный стековый слот для сохранения результата

```
val cachedLambda = null
val lambda = onStack( { x => println(x) } )
```

```
def evacuation[T](x: T): T = {
  ...
} else if (*(8x + 8) != null) {
  return *(8x + 8)
} else {
  ... // системный вызов

  *(8x + 8) = result
  return result
}
```

Окружение

Многоязыковая виртуальная машина Huawei

1. Оптимизирующий статический компилятор,
2. Управляемая среда исполнения,
3. Поддержка языков Java, Scala, Cangjie.



Результаты

Статистика

Количество перемещений на стек; больше — лучше

Название теста	Анализ утеканий	Межпроцедурный анализ частичных утеканий
scala-std	103	397
Виртуальная машина Huawei	394	4488

Заключение

В ходе работы было сделано:

1. Изучены существующие подходы к уменьшению издержек на выделение объектов в куче,
2. Разработана и реализована оптимизация «межпроцедурный анализ частичных утеканий»,
3. Полученное решение было протестировано.

Направления дальнейшей работы

1. Расширить анализ на другие типы, помимо лямбда-функций,
2. Добавить поддержку использования профилировочной информации.

Спасибо за внимание

Анализ утеканий

Алгоритм

NoEscape



RetEscape

RcvEscape



RetRcvEscape



GlobalEscape

Межпроцедурный анализ частичных эвакуаций

Алгоритм

